C# / .NET STUDY NOTES

Operators and Expressions in C#

Roadmap Topic 3

Learning focus
Expressions, arithmetic, assignment, comparison, Boolean logic, null handling, precedence, overflow, and practical Fundraising Platform rules.

Examples
Gift calculations, campaign goals, allocation validation, pagination, retry logic, authorization, and null-safe access.

Values / Variables → Operator → Expression → Result → Business Logic

Contents

01 Topic Coverage
02 What Is an Expression?
03 What Is an Operand?
04 What Is an Operator?
05 Arithmetic Operators
06 Assignment Operator =
07 Comparison Operators
08 Boolean Logical Operators
09 Short-Circuit Evaluation
10 Bitwise Operators
11 Shift Operators
12 Conditional Operator ?:
13 Null-Coalescing Operator ??
14 Null-Coalescing Assignment ??=
15 Null-Conditional Operator ?.
16 Operator Precedence
17 Associativity
18 Evaluation Order
19 Expressions vs Statements
20 checked and unchecked
21 Performance Considerations
22 Security Considerations
23 Interview Questions
24 Best Practices
25 Quick Revision
26 Complete Fundraising Example
27 Complete Topic Flow

Topic Coverage

Operators are the symbols and keywords that tell C# to perform an operation on one or more values.

Before learning conditions, loops, methods, LINQ, OOP, or ASP.NET Core logic, you need to understand how expressions are created and evaluated.

This topic covers:

- expressions;
- operands;
- operators;
- unary, binary, and ternary operators;
- arithmetic operators;
- integer and decimal arithmetic;
- division;
- remainder;
- increment and decrement;
- assignment operators;
- compound assignment;
- comparison operators;
- equality operators;
- Boolean logical operators;
- short-circuit operators;
- bitwise operators;
- shift operators;
- conditional `?:` operator;
- null-coalescing `??`;
- null-coalescing assignment `??=`;
- null-conditional `?.` and `?[]`;
- C# 14 null-conditional assignment;
- operator precedence;
- associativity;
- parentheses;
- evaluation order;
- `checked` and `unchecked`;
- overflow;
- expressions versus statements;
- practical Fundraising Platform examples;
- common mistakes;
- performance and correctness considerations;
- interview questions;
- practical exercises.

The basic flow is:

```
Values / Variables
        →
Operator
        →
Expression
        →
Result
        →
Used in Assignment / Condition / Method / Calculation
```

Example:

```
decimal giftAmount = 1000m;
decimal fee = 25m;

decimal netAmount = giftAmount - fee;
```

Here:

```
giftAmount
    →
operand

-
    →
operator

fee
    →
operand

giftAmount - fee
    →
expression

975
    →
result
```

C# provides operator groups for arithmetic, comparison, Boolean logic, assignment, null handling, member access, and many other operations. Operator precedence and associativity determine how larger expressions are interpreted.

What Is an Expression?

Definition

An **expression is a combination of values, variables, operators, method calls, or other expressions that produces a value**.

Example:

```
giftAmount - fee
```

This expression produces a value.

If:

```
giftAmount = 1000
fee        = 25
```

then:

```
giftAmount - fee
       →
975
```

Simple Expression Examples

```
10 + 20
```

produces:

```
30
```

```
giftAmount > 1000m
```

produces:

```
true
or
false
```

```
donorName ?? "Unknown"
```

produces either:

```
donorName
```

or:

```
"Unknown"
```

Expressions Can Be Nested

Example:

```
decimal finalAmount =
    giftAmount - (refundAmount + processingFee);
```

The inner expression:

```
refundAmount + processingFee
```

is evaluated first because of the parentheses.

Then:

```
giftAmount - result
```

is evaluated.

Flow:

```
refundAmount + processingFee
        →
subtotal
        →
giftAmount - subtotal
        →
finalAmount
```

What Is an Operand?

Definition

An **operand is a value on which an operator performs an operation**.

Example:

```
10 + 20
```

Here:

```
10
    →
left operand

+
    →
operator

20
    →
right operand
```

Another example:

```
giftAmount > 1000m
```

Operands:

```
giftAmount

1000m
```

Operator:

```
>
```

What Is an Operator?

Definition

An **operator tells C# what operation should be performed on its operands**.

Examples:

```
+
-
*
/
%
==
!=
>
<
&&
||
!
=
??
?.
```

Each operator has rules about:

- which operand types are allowed;
- what result type is produced;
- precedence;
- associativity;
- whether it short-circuits;
- whether overflow can occur.

Operator Categories by Number of Operands

Operators can also be classified based on the number of operands they use.

Unary Operator

A unary operator works on **one operand**.

Example:

```
int count = 10;

count++;
```

`++` works on:

```
count
```

one operand.

Other unary examples:

```
-count
```

```
!isActive
```

```
++count
```

```
--count
```

Binary Operator

A binary operator works on **two operands**.

Example:

```
giftAmount + additionalGift
```

Two operands:

```
giftAmount

additionalGift
```

Operator:

```
+
```

Most common operators are binary.

Ternary Operator

The conditional operator uses **three expressions**.

Syntax:

```
condition
    ? valueWhenTrue
    : valueWhenFalse
```

Example:

```
string giftCategory =
    giftAmount >= 1000m
        ? "Major Gift"
        : "Regular Gift";
```

Flow:

```
giftAmount >= 1000?
        →
true
        →
"Major Gift"

or

false
        →
"Regular Gift"
```

Arithmetic Operators

Arithmetic operators perform mathematical calculations.

The important operators are:

```
+
-
*
/
%
++
--
```

Microsoft groups `+`, `-`, `*`, `/`, `%`, `++`, and `--` as the primary arithmetic operators for numeric types.

Addition Operator +

Example:

```
decimal gift1 = 500m;
decimal gift2 = 300m;

decimal total =
    gift1 + gift2;
```

Result:

```
800
```

Fundraising use:

```
Gift 1
$500
    +
Gift 2
$300
    →
Total Giving
$800
```

Addition with Integers

```
int onlineGiftCount = 10;
int checkGiftCount = 5;

int totalGiftCount =
    onlineGiftCount + checkGiftCount;
```

Result:

```
15
```

String Concatenation with +

The `+` operator can also combine strings.

```
string firstName = "John";
string lastName = "Smith";

string displayName =
    firstName + " " + lastName;
```

Result:

```
John Smith
```

For short combinations this is fine.

For many repeated string modifications, later we will study:

```
StringBuilder
```

because strings are immutable.

Subtraction Operator -

Example:

```
decimal giftAmount = 1000m;
decimal refundedAmount = 250m;

decimal remainingAmount =
    giftAmount - refundedAmount;
```

Result:

```
750
```

Fundraising Allocation Example

```
decimal totalGift = 1000m;
decimal cancerResearch = 700m;

decimal remaining =
    totalGift - cancerResearch;
```

Result:

```
300
```

This could later be allocated to another fund.

Multiplication Operator *

Example:

```
decimal monthlyGift = 50m;
int months = 12;

decimal yearlyGiving =
    monthlyGift * months;
```

Result:

```
600
```

Flow:

```
Monthly Gift
$50
    ×
12 months
    →
Annual Giving
$600
```

Division Operator /

Division requires special attention because its behavior depends on operand types.

Example:

```
decimal total = 1000m;
decimal parts = 4m;

decimal result =
    total / parts;
```

Result:

```
250
```

Integer Division

This is extremely important.

```
int value = 5 / 2;
```

Result:

```
2
```

not:

```
2.5
```

Why?

Both operands are integers:

```
5
2
```

Therefore C# performs integer division.

Integer Division Flow

```
5 / 2
    →
both operands are int
    →
integer division
    →
fraction removed
    →
2
```

Getting Decimal Division

Use a decimal or floating-point operand.

```
decimal result =
    5m / 2m;
```

Result:

```
2.5
```

Or:

```
double result =
    5.0 / 2;
```

Result:

```
2.5
```

Fundraising Percentage Example

Suppose:

```
Goal Amount = $10,000

Raised      = $7,500
```

Correct:

```
decimal goalAmount = 10_000m;
decimal raisedAmount = 7_500m;

decimal percentage =
    raisedAmount / goalAmount * 100m;
```

Result:

```
75
```

meaning:

```
75%
```

Common Integer Division Mistake

Bad:

```
int raised = 7500;
int goal = 10000;

decimal percentage =
    raised / goal * 100;
```

Problem:

```
7500 / 10000
        →
integer division
        →
0

0 × 100
        →
0
```

Correct:

```
decimal percentage =
    (decimal)raised / goal * 100m;
```

Division by Zero

For integer division:

```
int value = 100 / 0;
```

causes:

```
DivideByZeroException
```

at runtime when the divisor is determined at runtime.

For business calculations, validate divisors.

Example:

```
if (goalAmount != 0)
{
    decimal percentage =
        raisedAmount / goalAmount * 100m;
}
```

Remainder Operator %

The `%` operator returns the remainder after division.

Example:

```
int result = 10 % 3;
```

Calculation:

```
10 / 3
    →
3 complete groups
    →
remainder 1
```

Result:

```
1
```

Common Uses of %

Examples:

```
Even / odd detection

Cycle logic

Grouping

Pagination calculations

Batch processing
```

Even Number Example

```
int giftId = 1002;

bool isEven =
    giftId % 2 == 0;
```

If remainder is zero:

```
even
```

Batch Processing Example

Suppose we process 100 gifts in batches of 25.

```
int totalGifts = 100;
int batchSize = 25;

int remaining =
    totalGifts % batchSize;
```

Result:

```
0
```

because 100 divides evenly into batches of 25.

Unary Plus +

Unary plus normally leaves the numeric value unchanged.

```
int value = 10;
int result = +value;
```

Result:

```
10
```

It is rarely needed in everyday business code.

Unary Minus -

Unary minus changes the sign.

```
decimal adjustment = 100m;

decimal reversedAdjustment =
    -adjustment;
```

Result:

```
-100
```

Be careful when negative values are not valid in the business domain.

Increment Operator ++

`++` adds one.

```
int giftCount = 10;

giftCount++;
```

Result:

```
11
```

Equivalent conceptually to:

```
giftCount =
    giftCount + 1;
```

Decrement Operator --

`--` subtracts one.

```
int retryCount = 3;

retryCount--;
```

Result:

```
2
```

Prefix vs Postfix Increment

This is an important interview topic.

Postfix

```
int x = 5;
int y = x++;
```

What happens?

```
y receives old x
        →
y = 5

then x increments
        →
x = 6
```

Final:

```
x = 6
y = 5
```

Prefix

```
int x = 5;
int y = ++x;
```

Flow:

```
increment x first
        →
x = 6

then assign
        →
y = 6
```

Final:

```
x = 6
y = 6
```

Prefix vs Postfix

```
x++
    →
use old value
then increment

++x
    →
increment first
then use new value
```

When increment is a standalone statement:

```
count++;
```

versus:

```
++count;
```

the practical result is normally the same.

The difference matters when the expression's value is used.

Assignment Operator =

The assignment operator stores a value in a variable.

```
decimal giftAmount = 1000m;
```

or later:

```
giftAmount = 1500m;
```

Do not confuse:

```
=
```

with:

```
==
```

`=` means assignment.

`==` means equality comparison.

Multiple Assignment

Assignments are right-associative.

Example:

```
int a;
int b;

a = b = 10;
```

C# interprets it as:

```
a = (b = 10)
```

So:

```
b = 10

then

a = 10
```

Assignment operators are right-associative according to C# operator rules.

Compound Assignment Operators

Instead of:

```
giftAmount =
    giftAmount + 100m;
```

you can write:

```
giftAmount += 100m;
```

Other examples:

```
+=
-=
*=
/=
%=
```

+=

```
decimal totalGiving = 1000m;

totalGiving += 250m;
```

Equivalent conceptually:

```
totalGiving =
    totalGiving + 250m;
```

Result:

```
1250
```

-=

```
decimal availableAmount = 1000m;

availableAmount -= 200m;
```

Result:

```
800
```

*=

```
decimal amount = 100m;

amount *= 2;
```

Result:

```
200
```

/=

```
decimal amount = 1000m;

amount /= 4;
```

Result:

```
250
```

%=

```
int value = 10;

value %= 3;
```

Result:

```
1
```

Why Compound Assignment Is Useful

It makes modifications concise.

Example:

```
totalRaised += newGiftAmount;
```

is often easier to read than:

```
totalRaised =
    totalRaised + newGiftAmount;
```

But avoid overly complex expressions.

Readable code is more important than writing the shortest code.

Comparison Operators

Comparison operators compare values.

Important operators:

```
>
<
>=
<=
==
!=
```

The result of a comparison is normally:

```
bool
```

meaning:

```
true
or
false
```

Greater Than >

```
decimal giftAmount = 1500m;

bool isMajorGift =
    giftAmount > 1000m;
```

Result:

```
true
```

Less Than <

```
decimal giftAmount = 100m;

bool belowMinimum =
    giftAmount < 500m;
```

Greater Than or Equal >=

```
bool majorGift =
    giftAmount >= 1000m;
```

This includes exactly:

```
1000
```

and anything greater.

Less Than or Equal <=

```
bool withinLimit =
    giftAmount <= 5000m;
```

Equality Operator ==

```
decimal amount = 1000m;

bool exactMatch =
    amount == 1000m;
```

Result:

```
true
```

Inequality Operator !=

```
string currencyCode = "USD";

bool isDifferentCurrency =
    currencyCode != "USD";
```

Result:

```
false
```

= vs ==

This is a basic but common mistake.

Assignment:

```
giftAmount = 1000m;
```

means:

```
store 1000
```

Comparison:

```
giftAmount == 1000m
```

means:

```
is giftAmount equal to 1000?
```

== and Equality

`==` behavior depends on the involved types.

For many built-in value types:

```
10 == 10
```

compares values.

For strings:

```
"John" == "John"
```

also performs value-based string equality.

For custom classes, equality behavior requires deeper understanding.

We will study:

```
==
.Equals()
ReferenceEquals()
value equality
reference equality
records
```

later under value/reference types and OOP.

Do not assume all reference types behave like strings.

Boolean Logical Operators

Boolean logic combines or modifies conditions.

Important operators:

```
&&
||
!
&
|
^
```

The most common in normal application logic are:

```
&&
||
!
```

Logical AND &&

`&&` means:

> Both conditions must be true.

Example:

```
bool canProcessGift =
    giftAmount > 0m
    &&
    isCampaignActive;
```

For `canProcessGift` to be true:

```
giftAmount > 0
        =
true

AND

isCampaignActive
        =
true
```

Both are required.

Truth Table for &&

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

Fundraising Example

```
bool isValidGift =
    giftAmount > 0m
    &&
    constituentId > 0;
```

Meaning:

```
Gift amount must be valid
AND
Constituent ID must be valid
```

Logical OR ||

`||` means:

> At least one condition must be true.

Example:

```
bool requiresReview =
    giftAmount >= 10_000m
    ||
    isAnonymous;
```

The gift requires review when:

```
large gift
OR
anonymous gift
```

Truth Table for ||

A	B	A || B
true	true	true
true	false	true
false	true	true
false	false	false

Logical NOT !

`!` reverses a Boolean value.

Example:

```
bool isActive = true;

bool isInactive =
    !isActive;
```

Result:

```
false
```

Fundraising example:

```
if (!isCampaignActive)
{
    Console.WriteLine(
        "Campaign is inactive.");
}
```

Meaning:

```
if campaign is NOT active
```

Short-Circuit Evaluation

This is extremely important.

The operators:

```
&&
||
```

are **short-circuiting** operators.

That means C# may not evaluate the second condition when the result is already known.

Short-Circuit with &&

Example:

```
if (gift is not null
    &&
    gift.TotalAmount > 0m)
{
}
```

If:

```
gift is null
```

then the first condition is:

```
false
```

For `&&`:

```
false && anything
        →
false
```

So C# does not need to evaluate:

```
gift.TotalAmount
```

This prevents a null access.

&& Flow

```
Condition A
    →
false?
    →
Yes
    →
stop
    →
overall result false

No
    →
evaluate Condition B
```

Short-Circuit with ||

Example:

```
if (isAdministrator
    ||
    HasGiftPermission(user))
{
}
```

If:

```
isAdministrator = true
```

then:

```
true || anything
        →
true
```

`HasGiftPermission` might not be called.

|| Flow

```
Condition A
    →
true?
    →
Yes
    →
stop
    →
overall result true

No
    →
evaluate Condition B
```

Why Short-Circuiting Matters

> It can improve:

```
Safety

Performance

Readability
```

Example:

```
if (campaign is not null
    &&
    campaign.IsActive)
{
}
```

Without short-circuiting, accessing `campaign.IsActive` while campaign is null would fail.

& vs &&

With Boolean operands:

```
&
```

can also perform logical AND.

But unlike:

```
&&
```

it evaluates both operands.

Example:

```
bool result =
    CheckA()
    &
    CheckB();
```

Both methods execute.

With:

```
bool result =
    CheckA()
    &&
    CheckB();
```

`CheckB()` may not execute.

| vs ||

Similarly:

```
|
```

evaluates both Boolean operands.

```
||
```

short-circuits.

For normal conditional logic:

```
&&
||
```

are usually what you want.

XOR Operator ^

For Boolean values, `^` means exclusive OR.

It is true when the operands are different.

Truth table:

A	B	A ^ B
false	false	false
false	true	true
true	false	true
true	true	false

Example:

```
bool exactlyOneSelected =
    useEmail ^ useSms;
```

This becomes true when exactly one option is true.

XOR is less common in ordinary business code than `&&` and `||`.

Bitwise Operators

Bitwise operators work on the individual bits of integer values.

Important ones include:

```
&
|
^
~
```

These are different from normal Boolean business logic, although some symbols overlap.

Bitwise AND &

Example:

```
int a = 6;
int b = 3;

int result =
    a & b;
```

Binary:

```
6 = 110
3 = 011

    110
AND 011
    ---
    010
```

Result:

```
2
```

Bitwise OR |

```
6 = 110
3 = 011

   110
OR 011
   ---
   111
```

Result:

```
7
```

Bitwise XOR ^

```
6 = 110
3 = 011

    110
XOR 011
    ---
    101
```

Result:

```
5
```

Bitwise Complement ~

`~` reverses the bits.

Example:

```
int value = 5;

int result =
    ~value;
```

This is mostly relevant to:

```
Flags

Low-level programming

Binary protocols

Performance-sensitive bit operations
```

not normal Fundraising calculations.

Flags Enum Preview

Bitwise operations become useful with `[Flags]` enums.

Example concept:

```
Read  = 001
Write = 010
Admin = 100
```

Permissions can be combined:

```
Read | Write
```

We will study enums and flags later.

Shift Operators

Shift operators move bits left or right.

Important operators:

```
<<
>>
>>>
```

They are typically used in:

```
Binary algorithms

Cryptography internals

Compression

Protocols

Low-level optimization

Bit flags
```

They are not everyday business-application operators.

Left Shift <<

Example:

```
int value = 1;

int result =
    value << 2;
```

Binary:

```
0001
    →
0100
```

Result:

```
4
```

For non-overflowing positive integers, shifting left by one is conceptually similar to multiplying by two.

But do not replace clear multiplication with bit shifting in normal business code.

Right Shift >>

```
int value = 8;

int result =
    value >> 1;
```

Result:

```
4
```

Again, this belongs mostly to specialized low-level scenarios.

Conditional Operator ?:

Definition

The conditional operator chooses one of two expressions based on a Boolean condition.

Syntax:

```
condition
    ? trueExpression
    : falseExpression
```

Example:

```
string status =
    giftAmount >= 1000m
        ? "Major"
        : "Standard";
```

Conditional Operator Flow

```
Condition
    →
true?
    →
True Expression

or

false
    →
False Expression
```

Fundraising Example

```
string donorDisplayName =
    isAnonymous
        ? "Anonymous Donor"
        : donorName;
```

This is very readable.

When to Use ?:

> Use it for **simple expression selection**.

Good:

```
string status =
    isActive
        ? "Active"
        : "Inactive";
```

Avoid deeply nested forms such as:

```
var result =
    a ? b ? c : d : e ? f : g;
```

Even if technically valid, it is difficult to understand.

Use normal `if`/`else` when logic becomes complex.

Null-Coalescing Operator ??

Definition

`??` returns the left operand when it is not null; otherwise it evaluates and returns the right operand.

Example:

```
string? donorName = null;

string displayName =
    donorName ?? "Unknown Donor";
```

Result:

```
Unknown Donor
```

Microsoft documents `??` as evaluating the fallback only when the left operand is null.

?? Flow

```
Left value
    →
null?
    →
No
    →
use left value

Yes
    →
evaluate and use right value
```

Fundraising Example

```
string? notes = null;

string displayNotes =
    notes ?? "No notes provided";
```

Right Side of ?? Is Evaluated Only When Needed

Example:

```
string value =
    donorName
    ??
    LoadDefaultDonorName();
```

If `donorName` is not null:

```
LoadDefaultDonorName()
```

is not called.

This can avoid unnecessary work.

Chaining ??

You can write:

```
string displayName =
    preferredName
    ??
    legalName
    ??
    "Unknown";
```

Flow:

```
preferredName available?
    →
Yes → use it

No
    →
legalName available?
    →
Yes → use it

No
    →
"Unknown"
```

The null-coalescing operator is right-associative.

Null-Coalescing Assignment ??=

Definition

`??=` assigns a value only when the current value is null.

Example:

```
List<long>? giftIds = null;

giftIds ??= new List<long>();
```

If `giftIds` is null:

```
new List<long>()
```

is assigned.

If it already has a list:

```
nothing changes
```

Microsoft describes `??=` as assigning the right-hand value only when the left-hand variable is null, and the right side is not evaluated otherwise.

??= Flow

```
Variable
    →
null?
    →
Yes
    →
evaluate right side
    →
assign it

No
    →
keep existing value
```

Lazy Initialization Example

```
List<long>? processedGiftIds = null;

processedGiftIds ??=
    new List<long>();

processedGiftIds.Add(1001);
```

You create the collection only when it is needed.

Null-Conditional Operator ?.

Definition

`?.` accesses a member only if the left-hand object is not null.

Example:

```
string? campaignName =
    gift.Campaign?.Name;
```

If:

```
gift.Campaign != null
```

then:

```
Name
```

is read.

If:

```
gift.Campaign == null
```

the whole expression returns:

```
null
```

instead of throwing a `NullReferenceException`.

Microsoft documents `?.` and `?[]` as null-conditional and short-circuiting operations.

?. Flow

```
Object
    →
null?
    →
Yes
    →
return null
    →
do not access member

No
    →
access member
```

Example Without ?.

Potentially dangerous:

```
string campaignName =
    gift.Campaign.Name;
```

If:

```
gift.Campaign
```

is null:

```
NullReferenceException
```

Safer Example

```
string? campaignName =
    gift.Campaign?.Name;
```

If there is no campaign:

```
campaignName = null
```

Combining ?. and ??

This is extremely common.

```
string campaignName =
    gift.Campaign?.Name
    ??
    "No Campaign";
```

Flow:

```
gift.Campaign
    →
exists?
    →
Yes
    →
read Name
    →
Name null?
        →
        No → use Name
        Yes → "No Campaign"

No
    →
"No Campaign"
```

Null-Conditional Indexer ?[]

Arrays and indexers also support null-conditional access.

Example:

```
string? firstItem =
    giftCodes?[0];
```

If `giftCodes` is null:

```
firstItem = null
```

If the array exists, index access occurs.

Important:

`?[]` protects against the collection itself being null.

It does **not** protect against an invalid index.

Microsoft notes that a non-null collection still throws its normal index-related exception if the supplied index is invalid.

Null-Conditional Chain

Example:

```
string? city =
    constituent
        ?.PrimaryAddress
        ?.City;
```

If any null-conditional step returns null, later null-conditional member access is skipped.

Conceptually:

```
Constituent
    →
exists?
    →
PrimaryAddress
    →
exists?
    →
City
```

Null-conditional operators short-circuit along such chains.

C# 14 Null-Conditional Assignment

Since the roadmap uses C# 14, there is an important modern addition.

C# 14 permits certain assignments through null-conditional access.

Example:

```
Campaign? campaign = GetCampaign();

campaign?.Description =
    "Updated description";
```

Meaning:

```
campaign != null
    →
perform assignment

campaign == null
    →
skip assignment
```

Microsoft documents null-conditional assignment as a C# 14 feature, including assignment to members and indexed elements when the target reference is non-null.

C# 14 Conditional Assignment Flow

```
campaign
    →
null?
    →
Yes
    →
skip assignment

No
    →
campaign.Description = newValue
```

The right-hand expression is only evaluated when the target is non-null.

When to Use Null-Conditional Assignment

Good:

```
campaign?.Description =
    updatedDescription;
```

when:

```
no campaign
```

legitimately means:

```
nothing to update
```

Do not use it when null represents an error that should be reported.

Example:

If campaign is required by the business rule, silently doing nothing may hide a bug.

Use:

```
if (campaign is null)
{
    throw new InvalidOperationException(
        "Campaign is required.");
}
```

when absence is invalid.

Null-Forgiving Operator !

This operator belongs primarily to nullability, but you should recognize it.

Example:

```
string name =
    maybeName!;
```

The postfix `!` tells the compiler:

> Trust me; I believe this expression is not null.

It does **not** add a runtime null check.

If the value is actually null and you later dereference it, the application can still fail.

Do not use `!` simply to silence warnings without understanding why the value is safe.

We will cover this deeply in the Nullability topic.

Operator Precedence

Definition

Operator precedence determines which operator binds/evaluates first when an expression contains different operators without explicit parentheses.

Example:

```
int result =
    2 + 3 * 4;
```

Multiplication has higher precedence than addition.

Therefore:

```
3 × 4
    →
12

2 + 12
    →
14
```

Result:

```
14
```

Microsoft's arithmetic operator documentation confirms that multiplicative operators have higher precedence than additive operators.

Parentheses Change Evaluation

```
int result =
    (2 + 3) * 4;
```

Now:

```
2 + 3
    →
5

5 × 4
    →
20
```

Result:

```
20
```

Microsoft recommends parentheses when you want to override or make precedence explicit.

Common Precedence Example

```
bool result =
    amount > 0m
    &&
    isActive
    ||
    isAdministrator;
```

This can be hard to read.

Even when you know precedence, clearer code may be:

```
bool result =
    (amount > 0m && isActive)
    ||
    isAdministrator;
```

Parentheses are not only for correctness.

They also improve readability.

Important Precedence Groups

You do not need to memorize the entire C# precedence table immediately.

Important general ordering includes:

```
Member / primary operations
        ↓
Unary operations
        ↓
Multiplication / Division / Remainder
        ↓
Addition / Subtraction
        ↓
Shift
        ↓
Relational comparisons
        ↓
Equality
        ↓
Bitwise operators
        ↓
&&
        ↓
||
        ↓
??
        ↓
?:
        ↓
Assignments
```

The exact full precedence table is documented by Microsoft.

For production code:

```
Do not rely on readers memorizing precedence.
Use parentheses when the intent is not immediately obvious.
```

Associativity

Definition

Associativity determines how operators of the same precedence are grouped.

Most binary operators are:

```
left-associative
```

Meaning:

```
left to right
```

Assignment, null-coalescing, and the conditional operator are important right-associative cases.

Left Associativity Example

```
int result =
    20 - 5 - 3;
```

Interpreted as:

```
(20 - 5) - 3
```

Calculation:

```
15 - 3
    →
12
```

Not:

```
20 - (5 - 3)
    →
18
```

Division Associativity Example

```
int result =
    20 / 5 / 2;
```

Interpreted:

```
(20 / 5) / 2
```

Result:

```
4 / 2
    →
2
```

Right Associativity of Assignment

```
a = b = c = 10;
```

Interpreted:

```
a = (b = (c = 10))
```

Right Associativity of ??

Example:

```
string name =
    preferredName
    ??
    legalName
    ??
    "Unknown";
```

Conceptually groups to the right.

But in ordinary usage, the useful mental model remains:

```
first non-null value wins
```

Evaluation Order

Precedence tells you **how expressions are grouped**.

Evaluation order tells you **when individual operands are actually evaluated**.

Do not confuse them.

Example:

```
int result =
    GetA() + GetB();
```

The operand expressions themselves are evaluated in language-defined order, while precedence determines grouping of operators.

This matters when expressions have side effects.

Avoid Hidden Side Effects

Bad style:

```
int result =
    x++ + ++x;
```

You may technically work out the answer, but the code is unnecessarily difficult to reason about.

Prefer simple statements:

```
x++;

x++;

int result = x;
```

or whatever logic actually represents the business requirement.

Professional code should optimize for understanding.

Expressions vs Statements

An **expression** produces a value.

Example:

```
giftAmount + fee
```

A **statement** represents an action in the program.

Example:

```
totalAmount = giftAmount + fee;
```

The right side:

```
giftAmount + fee
```

is an expression.

The entire line:

```
totalAmount = giftAmount + fee;
```

is an assignment statement.

Expression Statement

Some expressions can appear as statements.

Example:

```
giftCount++;
```

or:

```
ProcessGift();
```

They perform an action even though they are also expression forms internally.

---

checked and unchecked

Integer arithmetic can overflow.

Suppose:

```
int value = int.MaxValue;
```

Then:

```
value + 1
```

cannot fit inside an `int`.

checked

`checked` asks C# to detect applicable integral overflow.

Example:

```
checked
{
    int value = int.MaxValue;
    value++;
}
```

At runtime:

```
OverflowException
```

unchecked

```
unchecked
{
    int value = int.MaxValue;
    value++;
}
```

Allows unchecked overflow behavior.

For signed integers this can wrap around.

Overflow Flow

```
Integer operation
    →
result fits?
    →
Yes
    →
return result

No
    →
checked context?
        →
        Yes
        →
        OverflowException

        No
        →
        unchecked overflow behavior
```

When Overflow Matters

Possible scenarios:

```
Counters

Financial quantity calculations

Batch sizes

Offsets

Array indexes

Lengths

Security-sensitive calculations

Large imports
```

Do not assume:

```
int is large enough forever
```

for values that can grow without clear bounds.

decimal and Arithmetic Correctness

Financial calculations should normally use:

```
decimal
```

Example:

```
decimal grossGift = 1000m;
decimal fee = 25m;

decimal netGift =
    grossGift - fee;
```

Do not change financial code to `double` just because floating-point operations can be faster.

Correct representation matters more.

Arithmetic and Business Validation

This is an important distinction.

C# allows:

```
decimal giftAmount = -500m;
```

because:

```
-500
```

is a valid decimal value.

But the Fundraising domain may say:

```
Gift amount must be > 0
```

Therefore operators alone do not enforce business rules.

Example:

```
bool isValidGift =
    giftAmount > 0m;
```

Later, OOP/domain design will show how to enforce such rules inside objects and services.

Practical Fundraising Example — Gift Allocation

Requirement:

> A $1,000 gift must be allocated completely between funds.

```
decimal giftAmount = 1000m;

decimal researchAllocation = 700m;
decimal assistanceAllocation = 300m;

decimal totalAllocated =
    researchAllocation
    +
    assistanceAllocation;

bool fullyAllocated =
    totalAllocated == giftAmount;
```

Flow:

```
Research Allocation
$700
        +
Assistance Allocation
$300
        →
Total Allocated
$1000
        →
Compare with Gift
$1000
        →
true
```

More Complete Allocation Rule

```
bool validAllocation =
    researchAllocation > 0m
    &&
    assistanceAllocation > 0m
    &&
    researchAllocation
        + assistanceAllocation
        == giftAmount;
```

Meaning:

```
Research amount positive
        AND
Assistance amount positive
        AND
Sum equals Gift amount
```

Practical Example — Campaign Goal Progress

```
decimal goalAmount = 100_000m;
decimal totalRaised = 75_000m;

decimal progressPercentage =
    goalAmount > 0m
        ? totalRaised / goalAmount * 100m
        : 0m;
```

This combines:

```
>
?:
/
*
```

Flow:

```
Goal > 0?
    →
Yes
    →
Raised / Goal × 100

No
    →
0
```

Practical Example — Donor Display Name

```
string? preferredName = null;
string legalName = "John Smith";

string displayName =
    preferredName
    ??
    legalName;
```

Result:

```
John Smith
```

Practical Example — Optional Campaign

Suppose a Gift may or may not belong to a campaign.

```
string campaignName =
    gift.Campaign?.CampaignName
    ??
    "No Campaign";
```

This combines:

```
?.
??
```

and avoids an unnecessary explicit nested null check.

Practical Example — Authorization Logic Preview

Later ASP.NET Core authorization logic might involve:

```
bool canEditGift =
    isAdministrator
    ||
    (
        isFundraiser
        &&
        ownsGift
    );
```

Meaning:

```
Administrator
        OR
Fundraiser who owns gift
```

Parentheses make the intended authorization rule clear.

Practical Example — Pagination

Suppose:

```
int pageNumber = 3;
int pageSize = 25;
```

The number of rows to skip:

```
int skip =
    (pageNumber - 1)
    *
    pageSize;
```

Calculation:

```
(3 - 1)
    ×
25
    →
50
```

Later in LINQ:

```
query
    .Skip(skip)
    .Take(pageSize);
```

Operators are therefore foundational even for Web API pagination.

Practical Example — Retry Logic

```
int retryCount = 0;
const int maxRetries = 3;

bool canRetry =
    retryCount < maxRetries;
```

After failure:

```
retryCount++;
```

Later:

```
bool exhausted =
    retryCount >= maxRetries;
```

Operators appear everywhere in application logic.

Operator Overloading Preview

C# allows certain operators to be defined for custom types.

For example, built-in types already define what:

```
+
-
==
```

mean.

Custom types can sometimes overload operators too.

Example concept:

```
Money a;
Money b;

Money total =
    a + b;
```

Operator overloading is an advanced design feature.

Do not overload operators simply to make code look clever.

The operation should have obvious domain meaning.

We will revisit this under advanced C#/OOP when appropriate.

Common Mistake — Integer Division

Wrong expectation:

```
int result = 5 / 2;
```

Expecting:

```
2.5
```

Actual:

```
2
```

Correct when fractional result is needed:

```
decimal result =
    5m / 2m;
```

Common Mistake — = Instead of ==

Wrong:

```
// if (giftAmount = 1000m)
```

You intended comparison.

Correct:

```
if (giftAmount == 1000m)
{
}
```

Common Mistake — Using & Instead of &&

This:

```
conditionA & conditionB
```

evaluates both Boolean operands.

This:

```
conditionA && conditionB
```

short-circuits.

For most normal conditional logic:

```
&&
```

is the expected operator.

Common Mistake — Complex Ternary Expressions

Avoid:

```
string value =
    a
        ? b
            ? "X"
            : "Y"
        : c
            ? "Z"
            : "W";
```

This is difficult to read.

Use normal control flow when decision logic becomes complicated.

Common Mistake — Incorrect Null Hiding

Example:

```
campaign?.Close();
```

This is appropriate only when:

```
campaign missing
```

means:

```
nothing to do
```

If missing campaign is an invalid application state, silently ignoring it may hide a bug.

Common Mistake — Too Much Null-Forgiving !

Avoid:

```
gift!.Campaign!.Name!
```

just to remove compiler warnings.

This is effectively saying:

```
I guarantee every value is non-null.
```

If that assumption is wrong, runtime failures remain possible.

Fix the data/model design or perform appropriate checks.

Common Mistake — Depending on Hard-to-Remember Precedence

Instead of:

```
bool result =
    a || b && c && d || e;
```

prefer intentional grouping:

```
bool result =
    a
    ||
    (b && c && d)
    ||
    e;
```

Even if both versions mean the same thing, the second communicates intent better.

Common Mistake — Side Effects Inside Expressions

Avoid:

```
if (retryCount++ < maxRetries
    && ProcessGift())
{
}
```

This combines state mutation and decision logic.

Clearer:

```
bool canRetry =
    retryCount < maxRetries;

if (canRetry)
{
    retryCount++;

    ProcessGift();
}
```

Longer is sometimes better when it reduces mental complexity.

Common Mistake — Comparing Floating-Point Values for Exact Equality

For general `double` calculations, exact equality can be unreliable because of binary floating-point representation.

Example:

```
double a = 0.1 + 0.2;
double b = 0.3;
```

Depending on the values and calculation, exact equality may not represent the real mathematical intention.

For money:

```
use decimal
```

For scientific floating-point values:

```
use tolerance-based comparisons when appropriate
```

Common Mistake — Dividing Before Multiplying with Integers

Example:

```
int percent =
    completed / total * 100;
```

This may become zero because integer division happens first.

Safer:

```
decimal percent =
    (decimal)completed
    / total
    * 100m;
```

Common Mistake — Mixing Business Meaning

Example:

```
decimal result =
    giftAmount + giftCount;
```

This may compile if both are numeric-compatible after conversions, but the operation might make no business sense.

Type correctness does not automatically imply domain correctness.

Always ask:

```
What does this calculation mean?
```

Performance Considerations

Most basic operators are extremely inexpensive.

Do not micro-optimize:

```
+
-
*
&&
```

in normal application code.

Performance issues usually come from:

```
Database access

Network calls

Large allocations

LINQ over huge collections

Serialization

I/O

Poor algorithms

Lock contention
```

rather than ordinary arithmetic.

Short-Circuiting Can Avoid Work

This:

```
bool allowed =
    isAdmin
    ||
    ExpensivePermissionCheck();
```

does not call:

```
ExpensivePermissionCheck
```

when:

```
isAdmin = true
```

This can improve both correctness and performance.

Avoid Expensive Repeated Expressions

Instead of:

```
if (CalculateTotal(gifts) > 1000m
    &&
    CalculateTotal(gifts) < 5000m)
{
}
```

if calculation is expensive:

```
decimal total =
    CalculateTotal(gifts);

if (total > 1000m
    &&
    total < 5000m)
{
}
```

This is:

```
clearer
+
avoids repeated work
```

Security Considerations

> Operators themselves are not generally a security feature, but incorrect logic can create security bugs.

Example:

```
bool authorized =
    isAdmin
    ||
    isOwner
    &&
    isActive;
```

Because `&&` has higher precedence than `||`, this is grouped as:

```
isAdmin
OR
(isOwner AND isActive)
```

If your requirement was:

```
(isAdmin OR isOwner)
AND
isActive
```

the missing parentheses create a security logic error.

Correct:

```
bool authorized =
    (isAdmin || isOwner)
    &&
    isActive;
```

Authorization conditions should be made extremely explicit.

Business Rule Example

Suppose a refund is allowed when:

```
Payment succeeded
AND
Refund amount > 0
AND
Refund amount <= remaining refundable balance
```

Code:

```
bool canRefund =
    paymentSucceeded
    &&
    refundAmount > 0m
    &&
    refundAmount <= remainingBalance;
```

Flow:

```
Payment succeeded?
    →
Yes
    →
Refund > 0?
    →
Yes
    →
Refund <= remaining balance?
    →
Yes
    →
Allow Refund
```

This is how operators become real business logic.

Practical Exercise — Arithmetic

Given:

```
decimal giftAmount = 1000m;
decimal fee = 25m;
decimal refund = 100m;
```

Calculate:

```
net amount
```

Answer:

```
decimal netAmount =
    giftAmount
    - fee
    - refund;
```

Result:

```
875
```

Practical Exercise — Allocation Validation

```
decimal giftAmount = 1000m;
decimal allocationA = 600m;
decimal allocationB = 400m;
```

Write a Boolean expression confirming:

- both allocations are positive;
- the sum equals the gift.

Answer:

```
bool isValid =
    allocationA > 0m
    &&
    allocationB > 0m
    &&
    allocationA + allocationB == giftAmount;
```

Practical Exercise — Major Gift

Requirement:

```
Gift is major when amount >= $5,000.
```

Answer:

```
bool isMajorGift =
    giftAmount >= 5000m;
```

Practical Exercise — Anonymous Display Name

Requirement:

```
If donorName is null, display "Anonymous".
```

Answer:

```
string displayName =
    donorName ?? "Anonymous";
```

Practical Exercise — Optional Campaign

Requirement:

```
Return CampaignName if campaign exists;
otherwise return "No Campaign".
```

Answer:

```
string campaignName =
    campaign?.CampaignName
    ??
    "No Campaign";
```

Practical Exercise — Ternary

Requirement:

```
Amount >= 1000 → "Priority"
Otherwise      → "Standard"
```

Answer:

```
string category =
    giftAmount >= 1000m
        ? "Priority"
        : "Standard";
```

Practical Exercise — Precedence

What is:

```
int result =
    2 + 3 * 4;
```

Answer:

```
14
```

because:

```
*
```

has higher precedence than:

```
+
```

Practical Exercise — Parentheses

What is:

```
int result =
    (2 + 3) * 4;
```

Answer:

```
20
```

Practical Exercise — Short Circuit

```
Gift? gift = null;

bool valid =
    gift is not null
    &&
    gift.TotalAmount > 0m;
```

Does the second expression execute?

Answer:

```
No.
```

Because the first condition is false and `&&` short-circuits.

Practical Exercise — Prefix and Postfix

```
int x = 5;
int a = x++;
```

Result:

```
a = 5
x = 6
```

Now:

```
int y = 5;
int b = ++y;
```

Result:

```
b = 6
y = 6
```

Interview Questions

What is an operator in C#?

An operator is a symbol or keyword that performs an operation on one or more operands.

Examples:

```
+
-
*
/
==
&&
??
```

What is an expression?

An expression is a combination of values, variables, operators, and possibly method calls that evaluates to a value.

Example:

```
giftAmount - refundAmount
```

produces a numeric result.

What is an operand?

An operand is the value or expression an operator acts upon.

In:

```
a + b
```

`a` and `b` are operands.

What is the difference between unary, binary, and ternary operators?

Unary:

```
one operand
```

Example:

```
!isActive
```

Binary:

```
two operands
```

Example:

```
a + b
```

Ternary:

```
three expression parts
```

Example:

```
condition ? a : b
```

What is the difference between / and %?

`/` performs division.

`%` returns the remainder.

Example:

```
10 / 3 = 3

10 % 3 = 1
```

What is integer division?

When both operands are integer types, the fractional result is discarded.

Example:

```
int result = 5 / 2;
```

produces:

```
2
```

not `2.5`.

What is the difference between x++ and ++x?

Both increment `x`.

`x++` returns/uses the old value before incrementing.

`++x` increments first and returns/uses the new value.

What is the difference between = and ==?

`=` is assignment.

```
amount = 100m;
```

`==` checks equality.

```
amount == 100m;
```

What does && mean?

Logical AND.

Both operands must evaluate to true.

It also short-circuits if the left operand is false.

What does || mean?

Logical OR.

At least one operand must be true.

It short-circuits if the left operand is already true.

What is short-circuit evaluation?

It means C# avoids evaluating later Boolean expressions when the final result is already known.

Example:

```
gift is not null
&&
gift.TotalAmount > 0
```

If `gift` is null, the second expression is not evaluated.

Difference between && and &?

For Boolean operands:

`&&` is short-circuiting.

`&` evaluates both operands.

`&` also serves as a bitwise operator for integral types.

Difference between || and |?

For Boolean operands:

`||` short-circuits.

`|` evaluates both sides.

`|` is also a bitwise OR operator for integral types.

What is the conditional operator?

The conditional operator is:

```
?:
```

Example:

```
string status =
    isActive
        ? "Active"
        : "Inactive";
```

It chooses one of two expressions based on a Boolean condition.

What does ?? do?

The null-coalescing operator returns the left value if it is non-null.

Otherwise, it returns/evaluates the right value.

Example:

```
string name =
    donorName ?? "Unknown";
```

What does ??= do?

It assigns the right-hand value only when the left-hand variable is null.

Example:

```
items ??= new List<Gift>();
```

What does ?. do?

`?.` performs member access only when the left object is non-null.

If the object is null, the expression returns null rather than accessing the member.

What is new about null-conditional assignment in C# 14?

C# 14 permits assignment through eligible null-conditional member/index access.

Example:

```
campaign?.Name = "Annual Giving";
```

The assignment happens only when `campaign` is non-null.

What is operator precedence?

Operator precedence determines which operators bind more strongly in an expression.

Example:

```
2 + 3 * 4
```

evaluates multiplication before addition.

Result:

```
14
```

What is associativity?

Associativity determines grouping when operators have equal precedence.

Most binary C# operators associate left-to-right.

Assignment, conditional, and null-coalescing operators are important right-associative cases.

Why should parentheses be used even when precedence is known?

Because parentheses make business intent clearer and reduce mistakes.

This is especially important for:

```
authorization logic

validation rules

complex Boolean expressions
```

What is checked?

`checked` enables overflow detection for applicable integral arithmetic and conversions.

If an overflow occurs:

```
OverflowException
```

can be thrown.

What is unchecked?

`unchecked` allows applicable integral arithmetic overflow to proceed without checked overflow exceptions.

Why are operators important in real applications?

Operators implement everyday business rules such as:

```
money calculations

validation

authorization conditions

pagination

status selection

null fallback

comparison

counters

percentages
```

They are fundamental to almost every later C# topic.

Best Practices

Use:

```
decimal
```

for financial arithmetic.

Be careful with:

```
integer division
```

when fractional results are needed.

Use:

```
&&
||
```

for normal short-circuiting Boolean logic.

Use parentheses when they improve meaning.

Use:

```
??
?.
```

when null is an expected and valid state.

Do not use null operators to silently hide invalid application state.

Prefer:

```
simple readable expressions
```

over clever one-line expressions.

Avoid expressions with unnecessary side effects.

Do not rely on readers remembering complex operator precedence.

Keep authorization and financial expressions especially explicit.

Use:

```
checked
```

where overflow would cause a serious correctness problem.

Do not replace clear arithmetic with bit operations unless the domain genuinely requires bit-level processing.

Quick Revision

Operator	Meaning
`+`	Addition / string concatenation
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Remainder
`++`	Increment
`--`	Decrement
`=`	Assignment
`+=`	Add and assign
`-=`	Subtract and assign
`*=`	Multiply and assign
`/=`	Divide and assign
`==`	Equal
`!=`	Not equal
`>`	Greater than
`<`	Less than
`>=`	Greater than or equal
`<=`	Less than or equal
`&&`	Short-circuit AND
`	
`!`	Logical NOT
`&`	Bitwise AND / non-short-circuit Boolean AND
`|`	Bitwise OR / non-short-circuit Boolean OR
`^`	XOR
`~`	Bitwise complement
`<<`	Left shift
`>>`	Right shift
`?:`	Conditional operator
`??`	Null fallback

Operator	Meaning
`??=`	Assign if null
`?.`	Null-conditional member access
`?[]`	Null-conditional index access

Complete Fundraising Example

```
decimal giftAmount = 1_000m;
decimal processingFee = 25m;
decimal refundedAmount = 100m;

bool paymentSucceeded = true;
bool campaignActive = true;

string? donorPreferredName = null;
string legalName = "John Smith";

decimal netAmount =
    giftAmount
    - processingFee
    - refundedAmount;

bool validGift =
    giftAmount > 0m
    &&
    paymentSucceeded
    &&
    campaignActive;

string displayName =
    donorPreferredName
    ??
    legalName;

string giftCategory =
    giftAmount >= 1000m
        ? "Major Gift"
        : "Standard Gift";

Console.WriteLine(netAmount);
Console.WriteLine(validGift);
Console.WriteLine(displayName);
Console.WriteLine(giftCategory);
```

Business flow:

```
Gift
$1000
    →
Subtract Fee
$25
    →
Subtract Refund
$100
    →
Net Amount
$875
```

Validation:

```
Amount > 0
    AND
Payment succeeded
    AND
Campaign active
    →
Gift valid
```

Display:

```
Preferred Name exists?
    →
No
    →
Use Legal Name
```

Classification:

```
Amount >= $1000?
    →
Yes
    →
Major Gift
```

Complete Topic Flow

```
Values / Variables
        →
Operators
        →
Arithmetic
Comparison
Logical
Assignment
Null Handling
        →
Expressions Created
        →
Precedence Determines Grouping
        →
Associativity Resolves Equal Precedence
        →
Short-Circuiting May Skip Work
        →
Expression Produces Result
        →
Result Used By

Assignment
Condition
Loop
Method
Validation
Business Rule
API Logic
Authorization
Financial Calculation
```

Fundraising example:

```
GiftAmount
        →
> 0
        →
PaymentSucceeded
        →
&&
        →
CampaignActive
        →
&&
        →
Valid Gift?
        →
Yes
        →
Continue Processing
```

Operators may look like a basic C# topic, but they are the language foundation behind conditions, loops, validation, LINQ predicates, authorization rules, financial calculations, pagination, null handling, and almost every business rule you will later write in ASP.NET Core.

Next roadmap topic: Conditional Statements and Loops.